Подробное code review проекта Wallet API

Подробное code review проекта Wallet API

Этот документ написан как разбор проекта для человека, который не очень уверенно читает код и особенно не любит Docker. Цель отчёта не просто перечислить файлы, а объяснить, зачем существует каждая часть проекта, как всё это запускается, как проходит запрос внутри приложения и какие сильные стороны и риски я увидел во время review.

Что я сделалЯ прочитал структуру проекта, верхнеуровневые конфигурации, Docker-настройки, код приложения, миграции и тесты, а затем попробовал запустить тесты на практике.

Что я написалЯ подготовил понятный обзор архитектуры, папок, процессов запуска и работы кода, а также выделил реальные замечания code review, чтобы документ был полезен и технически, и организационно.

Дата анализа25.03.2026

Объект анализаНебольшой REST API на FastAPI и PostgreSQL для работы с кошельками: просмотр баланса и операции пополнения или списания.

1. Что это за проект простыми словами

Перед нами сервис кошельков. Он хранит деньги пользователей в базе данных и умеет делать две вещи: показывать текущий баланс кошелька и изменять его через пополнение или списание.

На человеческом языке логика такая: клиент отправляет запрос на сервер, сервер проверяет данные, находит нужный кошелёк в базе, аккуратно меняет баланс и возвращает результат. При этом автор проекта постарался защититься от самой опасной ситуации для кошельков: когда несколько операций идут одновременно и могут потерять деньги или испортить баланс.

Если совсем коротко: это не интернет-банк целиком, а аккуратно вырезанный кусок системы, который отвечает только за баланс одного кошелька и изменение этого баланса.

Основные технологии и зачем они здесь нужны

Технология | Что это значит простыми словами | Зачем используется здесь |

Python | Язык программирования | На нём написана вся логика сервиса |

FastAPI | Фреймворк для создания HTTP API | Принимает запросы и возвращает ответы |

PostgreSQL | База данных | Хранит таблицу кошельков и их баланс |

SQLAlchemy | Прослойка между Python и SQL | Позволяет работать с таблицами через Python-классы |

Alembic | Система миграций базы | Создаёт и изменяет структуру таблиц |

pytest | Фреймворк тестирования | Проверяет поведение API и конкурентные сценарии |

Docker | Упаковка приложения в контейнер | Позволяет запускать сервис и базу одинаково на разных машинах |

2. Как я проводил review

Хорошее code review это не чтение файлов по диагонали. Я шёл от внешнего слоя к внутреннему, чтобы понять не только синтаксис, но и реальный жизненный цикл приложения.

? Сначала я прочитал верхнеуровневые файлы: README.md, Dockerfile, docker-compose.yml, pyproject.toml, .env.example, alembic.ini.

? Потом я прошёл по папке app и проследил путь запроса: роуты, сервисы, репозитории, модели базы, конфигурация, обработка ошибок.

? После этого я посмотрел папку alembic, чтобы увидеть, как реально создаётся таблица в базе.

? Дальше я разобрал docker/entrypoint.sh, чтобы понять, что происходит внутри контейнера при старте.

? Затем я изучил тесты и попробовал их выполнить в реальном окружении.

? На финальном этапе я собрал не только описание проекта, но и замечания review: где код хорош, где есть риски и почему это важно.

Что именно я пишу в этом документе и зачем: я не переписываю код и не дублирую README, а создаю мост между исходниками и человеком. Такой документ помогает объяснить проект заказчику, коллеге, менеджеру или начинающему разработчику без необходимости читать весь код построчно.

3. Docker без сложных слов

Docker часто пугает, потому что кажется отдельным миром. На самом деле здесь он используется очень прикладно: чтобы поднять два изолированных процесса вместе и не заставлять человека руками настраивать всё окружение.

? Docker image — рецепт программы: какие файлы взять, что установить и какую команду запускать.

? Container — уже запущенный экземпляр этого рецепта.

? Dockerfile — инструкция, как собрать образ приложения.

? docker-compose.yml — инструкция, как поднять сразу несколько контейнеров и связать их между собой.

? Volume — постоянное хранилище, чтобы база данных не теряла данные после перезапуска.

В этом проекте Docker Compose поднимает два сервиса: db с PostgreSQL и app с самим API. Контейнер базы стартует первым, проходит healthcheck, и только потом запускается приложение. Это спасает от типичной ошибки, когда приложение стартует раньше базы и падает на первом же подключении.

Как читается Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY ...
RUN pip install --no-cache-dir .
ENTRYPOINT ["/app/docker/entrypoint.sh"]
```

По-человечески это значит: взять минимальный Linux с Python, положить внутрь проект, установить зависимости и запускать контейнер через стартовый скрипт.

Что делает docker/entrypoint.sh

```
python -m app.prestart
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Порядок очень правильный: сначала проверяется доступность базы, потом применяются миграции, и только после этого стартует веб-сервер.

4. Полный путь запуска проекта

? Человек запускает docker-compose up --build.

? Docker читает docker-compose.yml и видит два сервиса: базу и приложение.

? Для сервиса db скачивается образ PostgreSQL 16 Alpine.

? Для сервиса app собирается образ по инструкции из Dockerfile.

? База поднимается и начинает инициализацию.

? healthcheck проверяет, готова ли база отвечать на запросы.

? Когда база готова, стартует контейнер приложения.

? Внутри контейнера выполняется python -m app.prestart, затем alembic upgrade head, затем запускается uvicorn.

При локальном запуске без Docker человеку пришлось бы сделать всё это вручную: установить зависимости, поднять PostgreSQL, настроить переменные окружения и применить миграции. Поэтому Docker здесь нужен не для красоты, а для повторяемого и предсказуемого старта.

5. Подробный обзор каждой папки и ключевых файлов

Корень проекта

Корень проекта это место, откуда начинается знакомство с репозиторием. Здесь лежат файлы, которые объясняют, как проект запускать, собирать и настраивать.

? README.md — человекочитаемое описание проекта, его возможностей и способов запуска.

? pyproject.toml — зависимости и метаданные Python-пакета.

? Dockerfile — рецепт сборки контейнера приложения.

? docker-compose.yml — инструкция, как поднимать приложение вместе с базой.

? .env.example — пример переменных окружения.

? alembic.ini — конфигурация Alembic.

? .gitignore — список файлов и папок, которые обычно не должны попадать в Git.

Папка .idea

Это служебные настройки IDE JetBrains. Здесь нет бизнес-логики приложения. Файлы вроде workspace.xml, modules.xml и профили инспекций нужны редактору кода, а не самому сервису.

Папка .pytest_cache

Это технический кэш фреймворка pytest. Он нужен для ускорения повторных запусков тестов и не участвует в логике API.

Папка .venv

Это локальная виртуальная среда Python. Здесь лежат установленные зависимости проекта, чтобы они не смешивались с глобальным Python на компьютере.

Практически она важна так: обычный запуск pytest -q не нашёл пакет asgi_lifespan, а запуск .venv\Scripts\pytest.exe уже увидел все нужные библиотеки. Значит, для работы и проверки проекта нужно использовать именно эту среду или заранее ставить dev-зависимости.

Папка alembic

Это история изменений структуры базы данных.

? alembic/env.py — подготавливает запуск миграций и подставляет актуальный адрес базы.

? alembic/script.py.mako — шаблон для генерации новых миграций.

? alembic/versions/20260325_0001_create_wallets_table.py — конкретная миграция, создающая таблицу wallets.

Если сказать совсем просто: даже идеально написанная модель ещё не создаёт таблицу в реальной базе. Это делает именно миграция.

Папка app

Это главное сердце проекта. Здесь лежит весь код приложения. Внутри папка разделена на несколько слоёв, и это хороший архитектурный признак: разные задачи разведены по разным местам.

? app/main.py — собирает FastAPI-приложение, подключает роуты, логирование и обработчики ошибок.

? app/prestart.py — отдельный стартовый скрипт, который ждёт готовности базы.

? app/__init__.py — служебный файл пакета.

Подпапка app/api

Это входная дверь приложения. Сюда попадает внешний HTTP-запрос.

? app/api/router.py — собирает маршруты API и подключает версию v1.

? app/api/v1/wallets.py — описывает два маршрута: чтение баланса и выполнение операции по кошельку.

Здесь почти нет бизнес-логики. Роут принимает запрос, извлекает данные, вызывает сервис и возвращает ответ.

Подпапка app/core

Это инфраструктурный фундамент приложения.

? app/core/config.py — загружает настройки и формирует итоговый адрес базы данных.

? app/core/exceptions.py — описывает собственные ошибки приложения и превращает их в HTTP-ответы.

? app/core/logging.py — настраивает логи.

Этот слой не меняет деньги напрямую, но без него сервис был бы неудобен в запуске, диагностике и сопровождении.

Подпапка app/db

Этот слой отвечает за соединение с базой и описание таблиц.

? app/db/base.py — базовый класс для ORM-моделей.

? app/db/session.py — создаёт движок SQLAlchemy, фабрику сессий и dependency get_session.

? app/db/models/wallet.py — описание таблицы кошельков: идентификатор, баланс, время создания, время обновления.

Именно здесь Python-класс превращается в модель таблицы, с которой умеет работать база данных.

Подпапка app/repositories

Репозиторий это слой чтения и поиска данных. В проекте он компактный: файл app/repositories/wallet.py умеет искать кошелёк по id и, при необходимости, брать строку на блокировку через SELECT ... FOR UPDATE.

Практический смысл такой: во время изменения баланса система может забронировать строку кошелька, чтобы две параллельные операции не

испортили друг другу расчёты.

Подпапка app/services

Сервисный слой это место, где живёт бизнес-логика. Именно здесь код решает, можно ли списать деньги, как проверить тип операции и как изменить баланс.

В файле app/services/wallet.py реализована основная логика:

? проверка, что операция либо DEPOSIT, либо WITHDRAW;

? проверка, что сумма больше нуля;

? открытие транзакции;

? загрузка кошелька с блокировкой;

? увеличение или уменьшение баланса;

? запрет уйти в минус;

? сохранение результата.

Это один из самых важных файлов проекта, потому что именно он описывает правила денег.

Подпапка app/schemas

Схемы определяют форму входящих и исходящих данных API. В файле app/schemas/wallet.py описано, что приходит на вход при операции и что должно уйти в ответе клиенту.

Проще говоря, это договор между клиентом и сервером: какие поля обязательны и в каком виде они передаются.

Папка docker

Здесь лежит entrypoint.sh — сценарий запуска контейнера приложения. Это маленький, но стратегически важный файл: именно он задаёт правильный порядок старта базы, миграций и сервера.

Папка tests

Это тесты проекта. Они нужны, чтобы автоматически проверить, что API ведёт себя как ожидается.

? tests/conftest.py — общие фикстуры: подключение к тестовой базе, создание приложения, клиент для HTTP-запросов, очистка таблиц.

? tests/test_wallets.py — сценарии проверки: успешное чтение баланса, пополнение, списание, ошибки и параллельные операции.

Особенно ценно, что автор тестирует не только обычные случаи, но и конкурентные сценарии. Для кошельков это действительно важно.

Папка wallet_api.egg-info

Это техническая папка, которую Python создаёт при установке проекта как пакета. Она содержит метаданные: зависимости, список файлов и служебную информацию для упаковки.

Папки __pycache__

Эти каталоги появляются автоматически, когда Python компилирует исходники в более быстрый промежуточный вид. Они полезны машине, но почти бесполезны человеку при чтении проекта.

6. Как проходит один запрос внутри кода

Ниже я разбираю реальный путь запроса, чтобы было видно, как папки работают не по отдельности, а вместе.

Сценарий 1. Получение баланса кошелька

? Клиент вызывает GET /api/v1/wallets/{wallet_id}.

? Маршрут находится в app/api/v1/wallets.py.

? FastAPI подготавливает объект сессии базы данных через dependency get_session из app/db/session.py.

? Роут вызывает wallet_service.get_wallet(...) из app/services/wallet.py.

? Сервис обращается к репозиторию wallet_repository.get_by_id(...).

? Репозиторий делает запрос к таблице wallets.

? Если кошелёк найден, данные возвращаются наверх по стеку вызовов.

? Если кошелёк не найден, выбрасывается WalletNotFoundError, а обработчик ошибок превращает это в HTTP 404.

Сценарий 2. Пополнение или списание

? Клиент вызывает POST /api/v1/wallets/{wallet_id}/operation и передаёт JSON с типом операции и суммой.

? Роут принимает тело запроса и передаёт его в сервис.

? Сервис проверяет, что операция допустима и что сумма больше нуля.

? Сервис открывает транзакцию: все следующие действия должны выполниться как единое целое.

? Кошелёк читается из базы с FOR UPDATE. Это блокирует строку на время изменения.

? Если операция DEPOSIT, баланс увеличивается.

? Если операция WITHDRAW, код сначала проверяет, что денег хватает.

? Если денег не хватает, выбрасывается InsufficientFundsError, транзакция откатывается и клиент получает HTTP 409.

? Если всё в порядке, изменения сохраняются, транзакция завершается и клиент получает новый баланс.

Почему тут важен SELECT ... FOR UPDATE: если два человека одновременно попытаются изменить один и тот же кошелёк, база не даст им писать поверх друг друга. Одна операция дождётся завершения другой. Это очень полезная защита от race condition.

7. Что в проекте сделано хорошо

Плюс Чистое разделение слоёвРоуты, бизнес-логика, доступ к базе, конфигурация и схемы разложены по отдельным папкам. Такой код легче сопровождать, объяснять и расширять.

Плюс Защита от конкурентных конфликтовИспользование транзакции и FOR UPDATE показывает, что автор подумал о параллельных запросах, а не только о счастливом пути.

Плюс Защита на уровне базы данныхВ модели и миграции есть ограничение balance >= 0. Это хорошо, потому что правило баланс не должен быть отрицательным защищается не только в Python-коде, но и на стороне самой базы.

Плюс Автоматическое применение миграций при старте контейнераВ docker/entrypoint.sh сначала проверяется база, потом применяются миграции, и только после этого стартует сервер. Это уменьшает число ручных шагов и ошибок при развёртывании.

Плюс Наличие тестов на конкуренциюВ tests/test_wallets.py есть отдельные проверки параллельных пополнений и списаний. Для системы кошельков это особенно полезно.

8. Основные замечания code review

Ниже не просто пожелания, а реальные риски, которые могут проявиться в эксплуатации, автоматической проверке или сопровождении проекта.

P2 Тесты могут формально пройти, вообще не проверив проект

В tests/conftest.py:39-45 при недоступной PostgreSQL вызывается pytest.skip(...). На практике это означает, что вся интеграционная проверка превращается в 13 skipped, а команда всё равно завершается успешно. Я это подтвердил запуском: тесты действительно не упали, а были пропущены из-за недоступной базы.

Риск для команды простой: в CI можно случайно не поднять тестовую базу и получить зелёный прогон, хотя на самом деле код не был проверен.

P2 Тесты проверяют ORM-модели, а не реальные миграции

В tests/conftest.py:40-42 таблицы создаются через Base.metadata.create_all(), а не через alembic upgrade head. Такой подход проверяет, что модели SQLAlchemy согласованы между собой, но не гарантирует, что актуальная миграция в папке alembic/versions действительно приведёт базу к тому же состоянию.

Если однажды миграции разъедутся с моделями, приложение в тестах может выглядеть рабочим, а реальная база после развёртывания будет другой.

P2 Нет защиты от слишком больших сумм, выходящих за пределы BIGINT

Поле amount в app/schemas/wallet.py:15-17 принимает любой Python-целочисленный тип, а проверка в app/services/wallet.py:79-82 убеждается только в том, что сумма больше нуля. При этом баланс хранится в базе как BigInteger в app/db/models/wallet.py:22-23.

Если клиент отправит очень большое число, Python его примет, а PostgreSQL может уже не уместить это значение. Тогда вместо понятной 400-ошибки пользователь получит внутреннюю 500-ошибку.

9. Что показала практическая проверка

Я попробовал запустить тесты в реальном окружении и получил два важных наблюдения.

Наблюдение 1. Системный Python не видит dev-зависимости

Запуск pytest -q завершился ошибкой ModuleNotFoundError: No module named 'asgi_lifespan'. Это означает, что для разработки и проверки проекта важно использовать локальную виртуальную среду или заранее ставить dev-зависимости.

Наблюдение 2. Тесты через .venv стартуют, но вся проверка пропускается без базы

Запуск .venv\Scripts\pytest.exe -q -rs отработал без падения, но все 13 тестов были пропущены, потому что тестовая PostgreSQL была недоступна. Дополнительно был warning о том, что pytest не смог записать кэш в .pytest_cache из-за отказа в доступе.

Результат запуска через .venv:

```
13 skipped
```

Причина: PostgreSQL test database is not available

10. Общая оценка проекта простыми словами

Проект производит хорошее впечатление как учебный или тестовый сервис кошельков: архитектура аккуратная, слои разделены, транзакции и блокировки для денег продуманы, а Docker-конфигурация упрощает запуск.

Самая сильная сторона проекта — автор не ограничился операциями пополнения и списания, а подумал о конкурентном доступе. Для кошельков это один из самых важных моментов.

Самые важные зоны внимания — качество автоматической проверки и жёсткость валидации входных сумм. То есть фундамент у проекта хороший, но вокруг него ещё стоит подтянуть страховочную обвязку.

Если объяснять человеку совсем без кода: проект уже умеет правильно хранить и менять баланс, но ещё не до конца жёстко контролирует, в каких условиях он считается действительно проверенным и какие экстремальные значения ему вообще разрешено принимать.

11. Короткое резюме

Я разобрал проект как систему целиком: как он собирается, как запускается через Docker, как запрос идёт через API, сервисы и базу, что лежит в каждой папке и какие у этого решения сильные и слабые стороны. Итог такой: код организован чисто и логично, Docker настроен в целом правильно, логика кошелька реализована аккуратно, но для более надёжного проекта я бы усилил валидацию сумм и перестроил тестовый контур так, чтобы отсутствие тестовой базы не выглядело как успешная проверка.

Документ подготовлен как сопровождающий PDF-обзор по проекту Wallet API.